

DSAI 3202 – Parallel and distributed computing

Lab – 6: Distributed computing using mpi4py

1. Objectives:

- Develop Python programs that take advantage of multiple machines to run.

2. Tools and Concepts:

- Python: Programming language.
- Packages: multiprocessing, mpi4py, pandas.

3. Exercises in conjunction with the lecture

3.a. Square program

- Create a function `square` that computes the square number of a series of integers from 1 to n.
- The program must be parallel and uses mpi4py.
- Save the provided Python script as `calculate_squares.py`.

3.b. Environment Setup:

- Ensure `mpi4py` is installed on all targeted machines.
- Copy the ssh-id from the main machine to the others.
- Use SSH to access each machine.

3.c. Distribute the Program:

- Copy `calculate_squares.py` to each machine in the same directory path or use a shared file system.
- Create a Host File: On your main machine, create `machines.txt` listing the IP addresses of all

3.d. Run the Program:

- Execute the program across all machines with

3.e. Observe Results:

- The root process (rank 0) gathers all partial results, prints the size of final array of squares and the last square as well as the time taken.

3.f. Modify your program to compute the square up to $1e8$

3.g. Bonus (5% in the second assignment):

- You are given 300 to run your program. What is the highest square you can reach?
- Bonus is only given to the group with the highest square with all other results not lost.

4. Building an MPI Program for Virus Spread Simulation

4.a. Step 1: Initialize the MPI Environment

- Import the necessary libraries.
- Initialize the MPI communicator.
- Get the rank and size of the current process.

4.b. Step 2: Define Parameters

- Define the population size, chance of virus spread, and generate a random vaccination rate for each process.
- Example: `population_size = 100, spread_chance = 0.3, vaccination_rate = np.random.uniform(0.1, 0.5)`.

4.c. Step 3: Initialize the Population

- Initialize the population array with zeros representing uninfected individuals.
- Infect a small percentage of individuals at the start.
- Example:

```
1.     if rank == 0:
2.         infected_indices = np.random.choice(population_size,
                                                int(0.1 * population_size),
                                                replace=False)
3.         population[infected_indices] = 1
```

4.d. Step 4: Implement Virus Spread Function

- Create a function to simulate virus spread based on spread chance and vaccination rate.
- Update the population array based on infection rules.
- Example:

```
def spread_virus(population):
    new_population = population.copy()
    # Implement virus spread logic here
    return new_population
```

4.e. Step 5: Simulate Virus Spread

- Iterate over multiple time steps to simulate virus spread through the population.
- Exchange data between processes using MPI communication.
- Example:

```
for _ in range(10):
    population = spread_virus(population)
    if rank != 0:
        comm.send(population, dest=0)
    else:
        for i in range(1, size):
```

```
received_data = comm.recv(source=i)
population += received_data
```

4.f. Step 6: Calculate Infection Rate

- Calculate the infection rate for each process based on the final infected population count.
- Print the infection rate for each process.
- Example:

```
total_infected = np.sum(population)
infection_rate = total_infected / population_size
print(f"Process {rank} Infection Rate: {infection_rate}")
```

4.g. Step 7: Run and Experiment

- Run the program on multiple processors to observe different infection rates based on random vaccination rates assigned to each process.
- Experiment with changing parameters like spread chance and vaccination rates to observe their impact on virus spread dynamics.

=====

Adding the hostname tracking to the lecture code

```
from mpi4py import MPI
import socket # To get machine name
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
hostname = socket.gethostname() # Get the name of the machine running the process
if rank == 0:
    data = {'a': 7, 'b': 3.14}
    comm.send(data, dest=1, tag=11)
    print(f"Process {rank} on {hostname}: Sent data {data} to process 1")
elif rank == 1:
    data = comm.recv(source=0, tag=11)
    print(f"Process {rank} on {hostname}: Received data {data}")
```

- Which machine executed the print statement
- Which process (rank) printed the message

Example Scenario

Let's assume:

- Host machine (where you run mpirun) = host1
- Other machine (where process 1 runs) = host2

Expected Output on `host1` (where `mpirun` was run)

```
Process 0 on host1: Sent data {'a': 7, 'b': 3.14} to process 1
Process 1 on host2: Received data {'a': 7, 'b': 3.14}
```